Chapter 4 - Spring Data Access Framework

4.1 Objectives

- To introduce the Data Access framework of Spring
- To detail the JDBC and Hibernate Architecture
- To understand the Spring JDBC, Spring DAO and the Integration to Hibernate

4.2 Prerequisites

The reader of this document should have a good understanding of Java programming language, JDBC and Hibernate. Knowledge on the Spring Core Architecture, Dependency injection is also required.

4.3 Audience

This tutorial is designed for Java programmers with a need to understand how to leverage spring fram

ework supported features to access database by integrating it with JDBC and Hibernate framework.

4.4 Course Content

4.4.1 Introduction

Spring's data access framework addresses common difficulties developers face when working with databases in applications like establishing and working with connections, exception management and handling various types of Data Access related exceptions, etc.,. Support is provided for all popular data access frameworks in Java: JDBC, iBatis/MyBatis, Hibernate, JDO, JPA, Oracle TopLink, Apache OJB, and Apache Cayenne, among others. For all of these supported frameworks, Spring provides these features.

- Resource management - automatically acquiring and releasing database resources
- Exception handling - translating data access related exception to a Spring data access hierarchy
- Transaction participation - transparent participation in ongoing transactions
- Resource unwrapping - retrieving database objects from connection pool wrappers
- Abstraction for BLOB and CLOB handling

In this document spring integration with JDBC and Hibernate will be covered in detail.

4.4.2 Spring Data Access Integration Architecture

The following figure of the Spring framework's architecture has the Data Access and Integration layer highlighted. The individual sections of each of them are explained below.

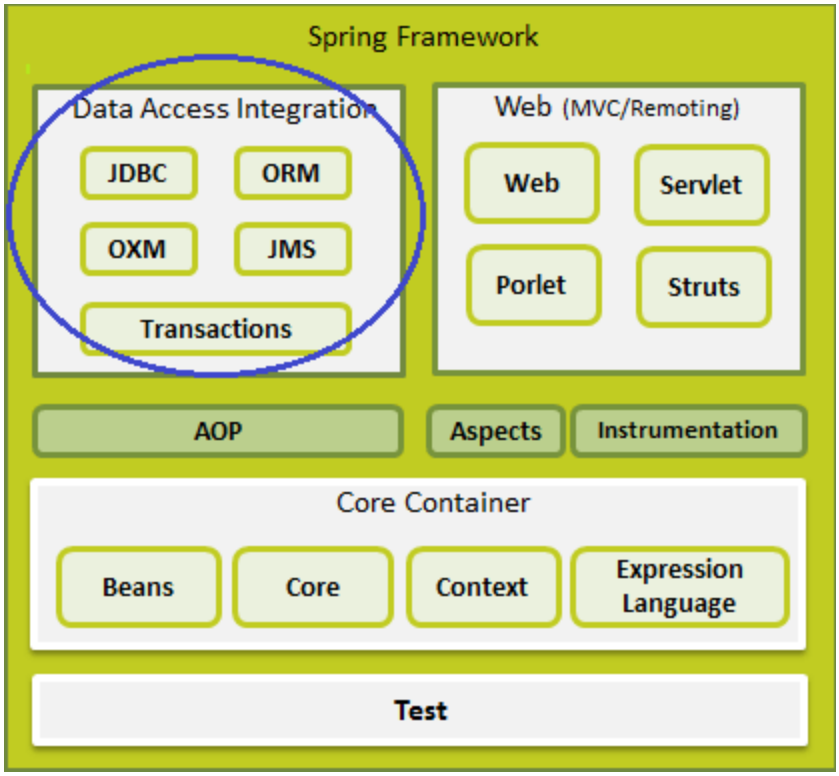

Figure 1: Spring Architecture

The Data Access/Integration layer consists of the JDBC, ORM, OXM, JMS and Transaction modules.

- The **JDBC** module provides a JDBC - Abstraction layer that removes the need to do tedious JDBC related coding.
- The **ORM** module provides integration layers for most frequently used object-relational mapping APIs, including JPA, JDO, Hibernate, and iBatis.
- The **OXM** module provides an abstraction layer that supports Object/XML mapping implementations for JAXB, Castor, XMLBeans, JiBX and Xstream.
- The Java Messaging Service **JMS** module contains features for producing and consuming messages.
- The **Transaction** module supports programmatic and declarative transaction management for classes that implement special interfaces and for all your POJOs.

4.4.3 Spring data access with JDBC Template

4.4.3.1 JDBC

JDBC stands for Java Database Connectivity, which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.

The JDBC library includes APIs for each of the tasks commonly associated with database usage:

- Making a connection to a database
- Creating SQL or MySQL statements
- Executing the formed SQL or MySQL queries in the database
- Viewing & Modifying the resulting records
- Fundamentally, JDBC is a specification that provides a complete set of interfaces that allows for portable access to an underlying database.

4.4.3.2 JDBC Architecture

The JDBC API supports both two-tier and three-tier processing models for database access but in general JDBC Architecture consists of two layers:

- **JDBC API**: This provides the application-to-JDBC Manager connection.
- **JDBC Driver API**: This supports the JDBC Manager-to-Driver Connection.
- **The JDBC API** uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.
- The **JDBC driver manager** ensures that the correct driver is used to access each data source. The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.

Following is the architectural diagram, which shows the location of the driver manager with respect to the JDBC drivers and the Java application:

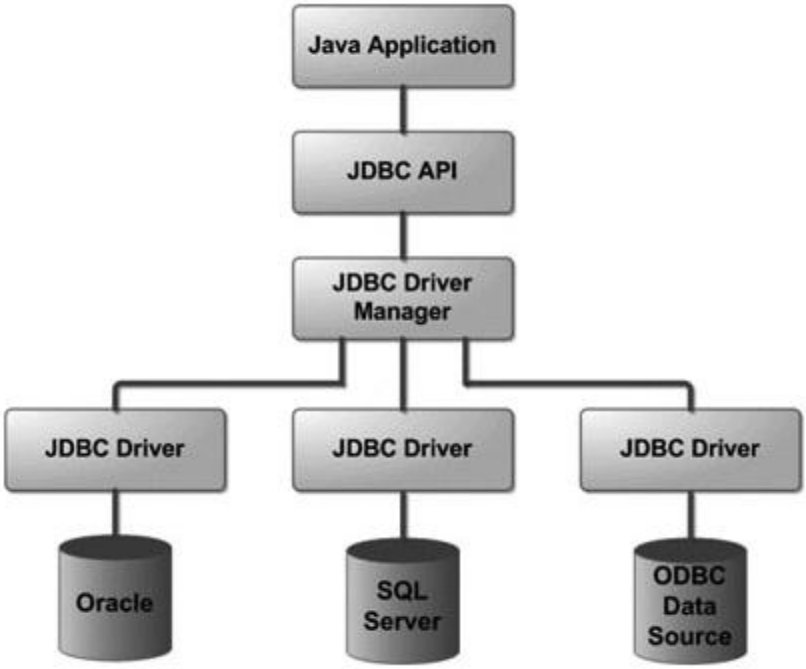

Figure 2: JDBC Architecture

4.4.3.3 Common JDBC Components:

The JDBC API provides the following interfaces and classes:

DriverManager: This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication sub-protocol. The first driver that recognizes a certain sub-protocol under JDBC will be used to establish a database Connection.

Driver: This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use Driver Manager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.

Connection : This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.

Statement : You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.

ResultSet: These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.

SQLException: This class handles any errors that occur in a database application.

4.4.3.4 Features of Spring JDBC

While working with database using plain old JDBC, it becomes cumbersome to write huge blocks of code to handle exceptions, opening and closing database connections etc. But Spring JDBC Framework takes care of all the low-level details starting from opening the connection, prepare and execute the SQL statement, process exceptions, handle transactions and finally close the connection. Hence it is enough to define the connection parameters, specify the SQL and do the iteration logic.

Handled by Spring JDBC	To be handled by Developer
• Open Connection • Preparation and Execution of SQL • Construct Iterator to loop the results (if any) • Process exceptions • Handle all transactions • Close the connection, statements and resultsets	• Specify the SQL and define the parameter values • Perform the logic within the iteration

Spring JDBC provides several approaches and correspondingly different classes to interface with the database. I'm going to take classic and the most popular approach which makes use of **JdbcTemplate** class of the framework. This is the central framework class that manages all the database communication and exception handling.

4.4.3.5 JDBCTemplate Class

The JdbcTemplate class executes SQL queries (insert and update statements) and stored procedure calls, performs iteration over ResultSets and extraction of returned parameter values. It also catches JDBC exceptions and translates them to the generic, more informative, exception hierarchy defined in the org.springframework.dao package.

Instances of the JdbcTemplate class are threadsafe once configured. So you can configure a single instance of a JdbcTemplate and then safely inject this shared reference into multiple DAOs.

A common practice when using the JdbcTemplate class is to configure a DataSource in your Spring configuration file, and then dependency-inject that shared DataSource bean into your DAO classes, and the JdbcTemplate is created in the setter for the DataSource.

4.4.3.6 Data Access Object (DAO)

DAO stands for data access object which is commonly used for database interaction. DAOs exist to provide a means to read and write data to the database and they should expose this functionality through an interface by which the rest of the application will access them.

The Data Access Object (DAO) support in Spring makes it easy to work with data access technologies like JDBC, Hibernate, JPA or JDO in a consistent way.

4.4.3.7 Spring JDBC Example - without JDBC Template

Step 1 : Create Customer model Objects

```
package com.customer.model;
import java.sql.Timestamp;
public class Customer
{
	int custId;
	String name;
	int age;
	//getter and setter methods
}
```

Step 2: Data Access Object

Step 2.1: Create Customer Dao interface.

```
package com.customer.dao;
import com.customer.model.Customer;
public interface CustomerDAO
{
	public void insert(Customer customer);
	public Customer findByCustomerId(int custId);
}
```

Step 2.2 : Create Customer Dao Implementation.

```
package com.customer.dao.impl;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import javax.sql.DataSource;
import com.customer.dao.CustomerDAO;
import com.customer.model.Customer;
public class JdbcCustomerDAO implements CustomerDAO
{
	private DataSource dataSource;
```

```
public void setDataSource(DataSource dataSource) {
        this.dataSource = dataSource;
}
public void insert(Customer customer){
        String sql = "INSERT INTO CUSTOMER " +
                        "(CUST_ID, NAME, AGE) VALUES (?, ?, ?)";
        Connection conn = null;
        try {
                conn = dataSource.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql);
                ps.setInt(1, customer.getCustId());
                ps.setString(2, customer.getName());
                ps.setInt(3, customer.getAge());
                ps.executeUpdate();
                ps.close();

        } catch (SQLException e) {
                throw new RuntimeException(e);

        } finally {
                if (conn != null) {
                        try {
                                conn.close();
                        } catch (SQLException e) {}
                }
        }
}
public Customer findByCustomerId(int custId){
        String sql = "SELECT * FROM CUSTOMER WHERE CUST_ID = ?";
        Connection conn = null;
        try {
                conn = dataSource.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql);
                ps.setInt(1, custId);
                Customer customer = null;
```

```
                        ResultSet rs = ps.executeQuery();
                        if (rs.next()) {
                                customer = new Customer(
                                        rs.getInt("CUST_ID"),
                                        rs.getString("NAME"),
                                        rs.getInt("Age")
                                );
                        }
                        rs.close();
                        ps.close();
                        return customer;
                } catch (SQLException e) {
                        throw new RuntimeException(e);
                } finally {
                        if (conn != null) {
                                try {
                                conn.close();
                                } catch (SQLException e) {}
                        }
                }
        }
}
```

Step 3: Configure beans classes (Spring-Customer.xml)

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">

        <bean id="customerDAO" class="com.customer.dao.impl.JdbcCustomerDAO">
                <property name="dataSource" ref="dataSource" />
        </bean>
        <bean id="dataSource"
                class="org.springframework.jdbc.datasource.DriverManagerDataSource">
                <property name="driverClassName" value="com.mysql.jdbc.Driver" />
```

```
                <property name="url" value="Provide Database URL" />
                <property name="username" value="Provide database User ID" />
                <property name="password" value="Provide database Password" />
        </bean>
</beans>
```

Step 4: Build the main application

```
package com.common;
 import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.customer.dao.CustomerDAO;
import com.customer.model.Customer;

public class App
{
   public static void main( String[] args )
   {
        ApplicationContext context =
                new ClassPathXmlApplicationContext("Spring-Module.xml");

      CustomerDAO customerDAO = (CustomerDAO) context.getBean("customerDAO");
      Customer customer = new Customer(1, "Ram",28);
      customerDAO.insert(customer);

      Customer customer1 = customerDAO.findByCustomerId(1);
      System.out.println(customer1);
    }
}
```

Output: Customer [age=28, custId=1, name=Ram]

4.4.3.8 Spring JDBC Example - with JDBC Template

Step 1 : Create Database table

```
CREATE TABLE Student(
   ID INT NOT NULL AUTO_INCREMENT,
   NAME VARCHAR(20) NOT NULL,
   AGE  INT NOT NULL,
   PRIMARY KEY (ID)
);
```

Step 2: Create Student bean

```
package com.sampleproject;
public class Student {
   private Integer age;
   private String name;
   private Integer id;

   public void setAge(Integer age) {
      this.age = age;
   }
   public Integer getAge() {
      return age;
   }

   public void setName(String name) {
      this.name = name;
   }
   public String getName() {
      return name;
   }

   public void setId(Integer id) {
      this.id = id;
```

```
  }
  public Integer getId() {
    return id;
  }
}
```

Step 3: Create mapping file

In this step we create a StudentMapper class that implements an interface called RowMapper. This interface has a single method called mapRow which helps to add the ResultSet into a collection. By this lot of coding is saved and this StudentMapper is invoked when the query is executed (using queryForObject method) by the JDBCTemplate.

```
package com.sampleproject;
import java.sql.ResultSet;
import java.sql.SQLException;
import org.springframework.jdbc.core.RowMapper;

public class StudentMapper implements RowMapper<Student> {
  public Student mapRow(ResultSet rs, int rowNum) throws SQLException {
    Student student = new Student();
    student.setId(rs.getInt("id"));
    student.setName(rs.getString("name"));
    student.setAge(rs.getInt("age"));
    return student;
  }
}
```

Step 4: Create DAO Interface

```
package com.sampleproject;
import java.util.List;
import javax.sql.DataSource;
public interface StudentDAO {
  public void setDataSource(DataSource ds);
  public void create(String name, Integer age);
  public Student getStudent(Integer id);
  public List<Student> listStudents();
  public void delete(Integer id);
```

```
   public void update(Integer id, Integer age);
}
```

Step 5: Create DAO Implementation

```
package com.sampleproject;
import java.util.List;
import javax.sql.DataSource;
import org.springframework.jdbc.core.JdbcTemplate;

public class StudentJDBCTemplate implements StudentDAO {
   private DataSource dataSource;
   private JdbcTemplate jdbcTemplateObject;
   public void setDataSource(DataSource dataSource) {
      this.dataSource = dataSource;
      this.jdbcTemplateObject = new JdbcTemplate(dataSource);
   }
   public void create(String name, Integer age) {
      String SQL = "insert into Student (name, age) values (?, ?)";
      jdbcTemplateObject.update( SQL, name, age);
      System.out.println("Created Record Name = " + name + " Age = " + age);
      return;
   }
   public Student getStudent(Integer id) {
      String SQL = "select * from Student where id = ?";
      Student student = jdbcTemplateObject.queryForObject(SQL,
                        new Object[]{id}, new StudentMapper());
      return student;
   }
   public List<Student> listStudents() {
      String SQL = "select * from Student";
      List <Student> students = jdbcTemplateObject.query(SQL,
                              new StudentMapper());
      return students;
   }
   public void delete(Integer id){
```

```
    String SQL = "delete from Student where id = ?";
    jdbcTemplateObject.update(SQL, id);
    System.out.println("Deleted Record with ID = " + id );
    return;
  }
  public void update(Integer id, Integer age){
    String SQL = "update Student set age = ? where id = ?";
    jdbcTemplateObject.update(SQL, age, id);
    System.out.println("Updated Record with ID = " + id );
    return;
  }
}
```

In the above step, we have created a DAO implementation class that uses a JDBCTemplate object to perform all database operations. The important methods of

- queryForObject – Executes a query as defined in the SQL and returns an object that gets mapped to a Rowmapper object as per the arguments in the associated method. Used in cases where result is a single row from database
- query – Executes a query as defined in the SQL and returns a collection that is generated after the resultset is mapped to a Rowmapper object. Used in cases where multiple rows are returned as a result of query execution.
- update – Performs the update operations to the data as defined in the SQL. Operations handled are insert, update and delete.

Step 6: Configure Spring beans (Beans.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
   http://www.springframework.org/schema/beans/spring-beans-3.0.xsd ">

  <!-- Initialization for data source -->
  <bean id="dataSource"
    class="org.springframework.jdbc.datasource.DriverManagerDataSource">
    <property name="driverClassName" value="com.mysql.jdbc.Driver"/>
    <property name="url" value="jdbc:mysql://localhost:3306/TEST"/>
    <property name="username" value="root"/>
    <property name="password" value="password"/>
```

```
   </bean>

   <!-- Definition for studentJDBCTemplate bean -->
   <bean id="studentJDBCTemplate"
      class="com.sampleproject.StudentJDBCTemplate">
      <property name="dataSource"  ref="dataSource" />
   </bean>

</beans>
```

Step 7: Create Main Application

```
package com.sampleproject;
import java.util.List;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.sampleproject.StudentJDBCTemplate;
public class MainApp {
   public static void main(String[] args) {
      ApplicationContext context =   new ClassPathXmlApplicationContext("Beans.xml");
     StudentJDBCTemplate studentJDBCTemplate =
           (StudentJDBCTemplate)context.getBean("studentJDBCTemplate");

      System.out.println("------Records Creation--------" );
      studentJDBCTemplate.create("Zara", 11);
      studentJDBCTemplate.create("Nuha", 2);
      studentJDBCTemplate.create("Ayan", 15);

      System.out.println("------Listing Multiple Records--------" );
      List<Student> students = studentJDBCTemplate.listStudents();
      for (Student record : students) {
         System.out.print("ID : " + record.getId() );
         System.out.print(", Name : " + record.getName() );
         System.out.println(", Age : " + record.getAge());
      }
```

```
    System.out.println("----Updating Record with ID = 2 -----" );
    studentJDBCTemplate.update(2, 20);

    System.out.println("----Listing Record with ID = 2 -----" );
    Student student = studentJDBCTemplate.getStudent(2);
    System.out.print("ID : " + student.getId() );
    System.out.print(", Name : " + student.getName() );
    System.out.println(", Age : " + student.getAge());

  }
}
```

Output:

```
------Records Creation--------
Created Record Name = Zara Age = 11
Created Record Name = Nuha Age = 2
Created Record Name = Ayan Age = 15
------Listing Multiple Records--------
ID : 1, Name : Zara, Age : 11
ID : 2, Name : Nuha, Age : 2
ID : 3, Name : Ayan, Age : 15
----Updating Record with ID = 2 -----
Updated Record with ID = 2
----Listing Record with ID = 2 -----
ID : 2, Name : Nuha, Age : 20
```

4.4.4 Spring data access with Hibernate

4.4.4.1 Hibernate Framework

Hibernate is a free, open source Java package that makes it easy to work with relational databases. Hibernate makes it seem as if your database contains plain Java objects like you use every day, without having to worry about how to get them out of (or back into) mysterious database tables. It liberates you to focus on the objects and features of your application, without having to

worry about how to store them or find them later. Hibernate's primary feature is mapping from Java classes to database tables (and from Java data types to SQL data types). Hibernate also provides data query and retrieval facilities. It generates SQL calls and relieves the developer from manual result set handling and object conversion.

4.4.4.2 Hibernate Mapping

Mapping Java classes to database tables is accomplished through the configuration of an XML file or by using Java Annotations. When using an XML file, Hibernate can generate skeleton source code for the persistence classes. This is unnecessary when annotations are used. Hibernate can use the XML file or the annotations to maintain the database schema.

4.4.4.3 Hibernate Architecture

Following figure is a very high level view of the Hibernate Application Architecture

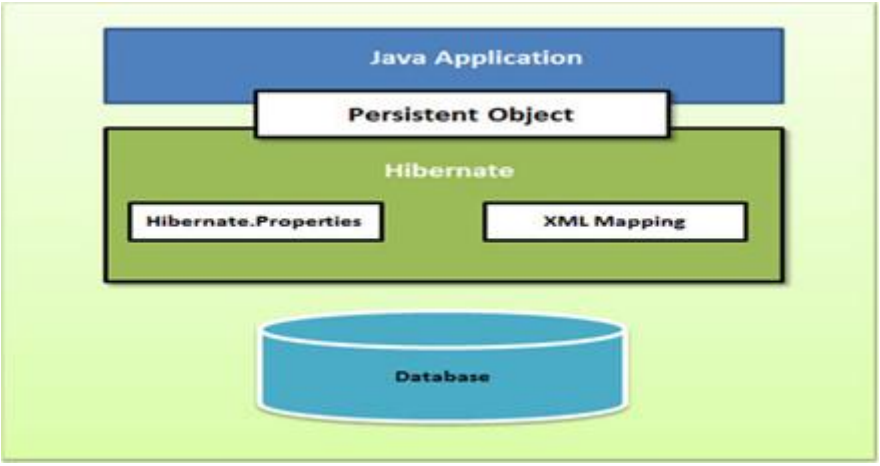

Figure 3: Hibernate Architecture

Hibernate uses various existing Java APIs, like JDBC, Java Transaction API(JTA), and Java Naming and Directory Interface (JNDI). JDBC provides a rudimentary level of abstraction of functionality common to relational databases, allowing almost any database with a JDBC driver to be supported by Hibernate. JNDI and JTA allow Hibernate to be integrated with J2EE application servers.

4.4.4.4 Features of Spring Hibernate integration

The Spring Framework provides integration with Hibernate, JDO, Oracle TopLink, iBATIS SQL Maps and JPA: in terms of resource management, DAO implementation support, and transaction strategies. For example for Hibernate, there is first-class support with lots of IoC convenience features, addressing many typical Hibernate integration issues.

Spring adds significant support when using the Object-Relational (O/R) mapping layer of your choice to create data access applications. First of all, you should know that once you started using Spring's support for O/R mapping, you don't have to go all the way. No matter to what extent, you're invited to review and leverage the Spring approach, before deciding to take the effort and risk of building a similar infrastructure in-house. Much of the O/R mapping support, no matter

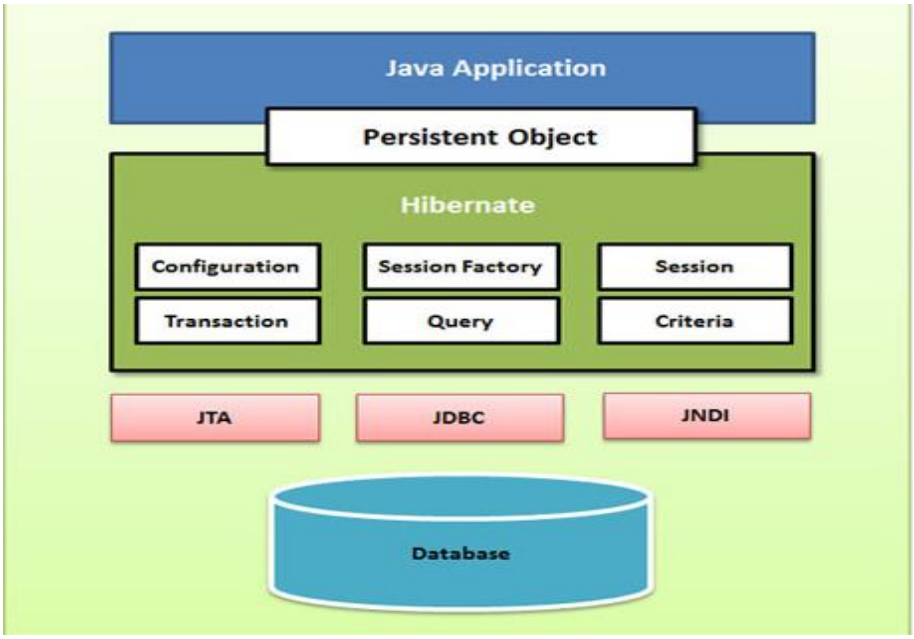

Figure 4: Spring Hibernate Integration

what technology you're using may be used in a library style, as everything is designed as a set of reusable JavaBeans. Usage inside a Spring IoC container does provide additional benefits in terms of ease of configuration and deployment; as such, most examples in this section show configuration inside a Spring container.

Some of the benefits of using the Spring Framework to create your ORM DAOs include:

- **Ease of testing.** Spring's IoC approach makes it easy to swap the implementations and config locations of Hibernate SessionFactory instances, JDBC DataSource instances, transaction managers, and mappes object implementations (if needed). This makes it much easier to isolate and test each piece of persistence-related code in isolation.
- **Common data access exceptions.** Spring can wrap exceptions from your O/R mapping tool of choice, converting them from proprietary (potentially checked) exceptions to a common runtime DataAccessException hierarchy. This allows you to handle most persistence exceptions, which are non-recoverable, only in the appropriate layers, without annoying boilerplate catches/throws, and exception declarations. You can still trap and handle exceptions anywhere you need to. Remember that JDBC exceptions (including DB specific dialects) are also converted to the same hierarchy, meaning that you can perform some operations with JDBC within a consistent programming model.
- **General resource management.** Spring application contexts can handle the location and configuration of Hibernate SessionFactory instances, JDBC DataSource instances, iBATIS SQL Maps configuration objects, and other related resources. This makes these values easy to manage and change. Spring offers efficient, easy and safe handling of persistence resources. For example: related code using Hibernate generally needs to use the same Hibernate Session for efficiency and proper transaction handling. Spring makes it easy to transparently create and bind a Session to the current thread, either by using an explicit 'template' wrapper class at the Java code level or by exposing a current Session through the Hibernate SessionFactory (for DAOs based on plain Hibernate API). Thus Spring solves many of the issues that repeatedly arise from typical Hibernate usage, for any transaction environment (local or JTA).
- **Integrated transaction management.** Spring allows you to wrap your O/R mapping code with either a declarative, AOP style method interceptor, or an explicit 'template' wrapper class at the Java code level. In either case, transaction semantics are handled for you, and proper transaction handling (rollback, etc) in case of exceptions is taken care of. As discussed below, you also get the benefit of being able to use and swap various transaction

managers, without your Hibernate/JDO related code being affected: for example, between local transactions and JTA, with the same full services (such as declarative transactions) available in both scenarios. As an additional benefit, JDBC-related code can fully integrate transactionally with the code you use to do O/R mapping. This is useful for data access that's not suitable for O/R mapping, such as batch processing or streaming of BLOBs, which still needs to share common transactions with ORM operations.

4.4.4.5 Spring Hibernate example 1

```
package com.sample.spring.hibernate;
import java.util.Collection;
public interface StudentDao
{
	public Student getStudent(long id);
	public Collection getAllStudents();
	public Collection getGraduatedStudents();
	public Collection findStudents(String lastName);
	public void saveStudent(Student std);
	public void removeStudent(Student std);
}
```

The following code shows the DAO implementation for this DAO interface, which now uses Spring to interact with Hibernate:

```
package com.sample.spring.hibernate;
import org.springframework.orm.hibernate4.support.HibernateDaoSupport;
import java.util.Collection;
public class HibernateStudentDao extends HibernateDaoSupport implements StudentDao {
	public Student getStudent(long id) {
		return (Student) getHibernateTemplate()
				.get(Student.class, new Long(id));
	}

	public Collection getAllStudents() {
		return getHibernateTemplate().find(
				"from Student std order by std.lastName, std.firstName");
```

```
        }

        public Collection getGraduatedStudents() {
                return getHibernateTemplate().find(
                                "from Student std where std.status=1");
        }

        public Collection findStudents(String lastName) {
                return getHibernateTemplate().find(
                                "from Student std where std.lastName like ?", lastName + "%");
        }

        public void saveStudent(Student std) {
                getHibernateTemplate().saveOrUpdate(std);
        }

        public void removeStudent(Student std) {
                getHibernateTemplate().delete(std);
        }
}
```

The following code shows the DAO class, which now uses HibernateTemplate directly. Note that this approach is not recommended because it gets you involved with the HibernateTemplate object in both the DAO class and the DAO configuration in the Spring context:

```
package com.sample.spring.hibernate;
import org.springframework.orm.hibernate3.HibernateTemplate;
import java.util.Collection;

public class HibernateStudentDao implements StudentDao {
        HibernateTemplate hibernateTemplate;

        public Student getStudent(long id) {
                return (Student) getHibernateTemplate()
                                .get(Student.class, new Long(id));
        }
```

```
	public Collection getAllStudents() {
		return getHibernateTemplate().find(
				"from Student std order by std.lastName, std.firstName");
	}

	public Collection getGraduatedStudents() {
		return getHibernateTemplate().find(
				"from Student std where std.status=1");
	}

	public Collection findStudents(String lastName) {
		return getHibernateTemplate().find(
				"from Student std where std.lastName like " + lastName + "%");
	}

	public void saveStudent(Student std) {
		getHibernateTemplate().saveOrUpdate(std);
	}
	public void removeStudent(Student std) {
		getHibernateTemplate().delete(std);
	}
	public HibernateTemplate getHibernateTemplate() {
		return hibernateTemplate;
	}

	public void setHibernateTemplate(HibernateTemplate hibernateTemplate) {
		this.hibernateTemplate = hibernateTemplate;
	}
}
```

Moreover, the DAO class can abandon the HibernateTemplate class and use the SessionFactory instance directly to interact with Hibernate. The following code shows HibernateStudentDao, which now works directly with the SessionFactory object:

```
package com.sample.spring.hibernate;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.Query;
import org.hibernate.SessionFactory;
import org.springframework.orm.hibernate4.SessionFactoryUtils;
import java.util.Collection;

public class HibernateStudentDao implements StudentDao {
	SessionFactory sessionFactory;

	public Student getStudent(long id) {
		Session session = SessionFactoryUtils.getSession(this.sessionFactory,
				true);
		try {
			return (Student) session.get(Student.class, new Long(id));
		} catch (HibernateException ex) {
			throw SessionFactoryUtils.convertHibernateAccessException(ex);
		} finally {
			SessionFactoryUtils.closeSession(session);
		}
	}

	public Collection getAllStudents() {
		Session session = SessionFactoryUtils.getSession(this.sessionFactory,
				true);
		try {
			Query query = session
				.createQuery("from Student std order by std.lastName, std.firstName");
			Collection allStudents = query.list();
			return allStudents;
		} catch (HibernateException ex) {
			throw SessionFactoryUtils.convertHibernateAccessException(ex);
		} finally {
			SessionFactoryUtils.closeSession(session);
```

```
		}
	}

	public Collection getGraduatedStudents() {
		Session session = SessionFactoryUtils.getSession(this.sessionFactory,
				true);
		try {
			Query query = session
				.createQuery("from Student std where std.status=1");
			Collection graduatedStudents = query.list();
			return graduatedStudents;
		} catch (HibernateException ex) {
			throw SessionFactoryUtils.convertHibernateAccessException(ex);
		} finally {
			SessionFactoryUtils.closeSession(session);
		}
	}
	public Collection findStudents(String lastName) {
		Session session = SessionFactoryUtils.getSession(this.sessionFactory,
				true);
		try {
			Query query = session
				.createQuery("from Student std where std.lastName like ?");
			query.setString(1, lastName + "%");
			Collection students = query.list();
			return students;
		} catch (HibernateException ex) {
			throw SessionFactoryUtils.convertHibernateAccessException(ex);
		} finally {
			SessionFactoryUtils.closeSession(session);
		}
	}
	public void saveStudent(Student std) {
		Session session = SessionFactoryUtils.getSession(this.sessionFactory,
				true);
```

```
		try {
			session.save(std);
		} catch (HibernateException ex) {
			throw SessionFactoryUtils.convertHibernateAccessException(ex);
		} finally {
			SessionFactoryUtils.closeSession(session);
		}
	}
	public void removeStudent(Student std) {
		Session session = SessionFactoryUtils.getSession(this.sessionFactory,
				true);
		try {
			session.delete(std);
		} catch (HibernateException ex) {
			throw SessionFactoryUtils.convertHibernateAccessException(ex);
		} finally {
			SessionFactoryUtils.closeSession(session);
		}
	}

	public void setSessionFactory(SessionFactory sessionFactory) {
		this.sessionFactory = sessionFactory;
	}
}
```

In all of the methods above, the SessionFactoryUtils class is used to obtain a Session object. The provided Session object is then used to perform the persistence operation. SessionFactoryUtils is also used to translate HibernateException to DataAccessException in the catch blocks and close the Session objects in the final blocks. Note that this DAO implementation bypasses the advantages of HibernateDaoSupport and HibernateTemplate. You must manage Hibernate's Session manually (as well as exception translation and transaction management) and implement much boilerplate code.

org.springframework.orm.hibernate4.SessionFactoryUtils is a Spring helper class for obtaining Session, reusing Session within transactions, and translating HibernateException to the generic DataAccessException.

This way is absolutely not the way to work with Session in Spring. Always use the HibernateTemplate class to work with Session objects behind the scenes as discussed earlier in a example which does not use session objects directly.

4.4.4.6 Spring Hibernate example 2

Step 1: Create model class

```
package com.sample.stock.model;

import java.io.Serializable;

public class Stock implements Serializable {

	private static final long serialVersionUID = 1L;

	private Long stockId;
	private String stockCode;
	private String stockName;

	//getter and setter methods...
}
```

Step 2: Create dao interface class

```
package com.sample.stock.dao;

import com.sample.stock.model.Stock;

public interface StockDao {

	void save(Stock stock);
	void update(Stock stock);
```

```
	void delete(Stock stock);
	Stock findByStockCode(String stockCode);

}
```

Step 3: Create dao implementation class

```
package com.sample.stock.dao.impl;

import java.util.List;
import org.springframework.orm.hibernate3.support.HibernateDaoSupport;
import com.sample.stock.dao.StockDao;
import com.sample.stock.model.Stock;

public class StockDaoImpl extends HibernateDaoSupport implements StockDao{

	public void save(Stock stock){
		getHibernateTemplate().save(stock);
	}

	public void update(Stock stock){
		getHibernateTemplate().update(stock);
	}

	public void delete(Stock stock){
		getHibernateTemplate().delete(stock);
	}

	public Stock findByStockCode(String stockCode){
		List list = getHibernateTemplate().find(
		"from Stock where stockCode=?",stockCode
	    );
		return (Stock)list.get(0);
	}
}
```

Step 4: Create hibernate mapping file

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
    <class name="com.sample.stock.model.Stock" table="stock">
        <id name="stockId" type="java.lang.Long">
            <column name="STOCK_ID" />
            <generator class="identity" />
        </id>
        <property name="stockCode" type="string">
            <column name="STOCK_CODE" length="10" not-null="true" unique="true" />
        </property>
        <property name="stockName" type="string">
            <column name="STOCK_NAME" length="20" not-null="true" unique="true" />
        </property>
    </class>
</hibernate-mapping>
```

Step 5: Create database property file

```
jdbc.driverClassName=com.mysql.jdbc.Driver
jdbc.url=PROVIDE YOUR CONNECTION URL HERE
jdbc.username=PROVIDE YOUR USER ID HERE
jdbc.password=PROVIDE YOUR PASSWORD HERE
```

This property file needs to be placed in a separate folder in the classpath. The path of the same is to be referred in the configuration file

Step 6: Configure sessionfactory, datasource and mapping files in Spring configuration file

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-2.5.xsd">
```

```
<bean
class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
        <property name="location">
                <value>properties/database.properties</value>
        </property>
</bean>

<bean id="dataSource"
      class="org.springframework.jdbc.datasource.DriverManagerDataSource">
        <property name="driverClassName" value="${jdbc.driverClassName}" />
        <property name="url" value="${jdbc.url}" />
        <property name="username" value="${jdbc.username}" />
        <property name="password" value="${jdbc.password}" />
</bean>


<bean id="sessionFactory"
   class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">

  <property name="dataSource">
    <ref bean="dataSource"/>
  </property>

  <property name="hibernateProperties">
    <props>
      <prop key="hibernate.dialect">org.hibernate.dialect.MySQLDialect</prop>
      <prop key="hibernate.show_sql">true</prop>
    </props>
   </property>

   <property name="mappingResources">
        <list>
       <value>/hibernate/Stock.hbm.xml</value>
        </list>
```

```
    </property>

</bean>

<bean id="stockDao" class="com.sample.stock.dao.impl.StockDaoImpl" >
        <property name="sessionFactory" ref="sessionFactory"></property>
</bean>

</beans>
```

Step 7: Create Main application to run

```
package com.sample.common;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.sample.stock.dao.StockDao;
import com.sample.stock.model.Stock;

public class App
{
    public static void main( String[] args )
    {
        ApplicationContext appContext =            new
        ClassPathXmlApplicationContext("spring/config/BeanLocations.xml");
        StockDao stockDao = (StockDao)appContext.getBean("stockDao");

        /** insert **/
        Stock stock = new Stock();
        stock.setStockCode("7668");
        stock.setStockName("HAIO");
        stockDao.save(stock);

        /** select **/
        Stock stock2 = stockDao.findByStockCode("7668");
```

```
        System.out.println(stock2);

        /** update **/
        stock2.setStockName("HAIO-1");
        stockDao.update(stock2);

        /** delete **/
        stockDao.delete(stock2);

        System.out.println("Done");
    }
}
```